Watch, Plan, Act

Diffusion Policies for Robot Imitation Learning | A guided project report

Can a robot learn a precise manipulation task from human examples? That is the question behind this project. A simulated Franka Panda must grasp a square nut, align it, and fit it onto a peg. It sees the scene through cameras and decides how to move its gripper.

The result at a glance
At 100,000 updates: **20% success for single-action BC, 48% for diffusion noise prediction, and 52% for diffusion clean-action prediction.** Each score comes from 25 simulator episodes.

Why learn from demonstrations?

Rather than hand-code every grasp and alignment correction, we show the model what a human did in each situation. It learns a mapping from the current observation to a robot command. This is imitation learning: the training targets come from demonstrations, not from optimizing evaluation rewards.

What is in one training example?

What the robot sees
Two 84 x 84 RGB images: external and wrist views.
End-effector position and quaternion orientation.
Gripper position.

What the human commanded
A 7-dimensional action:
3 position increments
3 rotation increments (axis-angle)
1 gripper command

The local Robomimic square/ph/image dataset contains **200 proficient-human demonstrations and 30,154 time steps**. Only the selected camera and robot-state fields are used. The policy does not receive the stored ground-truth object state.

Project context. This work is based on CSC2626: Imitation Learning for Robotics. The course supplied the network and simulator scaffolding. The implemented algorithm components include DDPM noising, both prediction objectives, reverse sampling, and action chunk execution. This report explains the resulting system and analyzes its saved evaluations.

All robot experiments discussed here use simulation. No physical robot deployment is claimed.

One view, two policies

Both methods use the same observation and backbone architecture, with separate learned weights.

Observe
External + wrist images
End-effector pose
Gripper position

→

Encode
ResNet18 image features
+ robot state
256-d condition vector

→

Choose actions
DiT1d backbone
BC: one action
Diffusion: short plan

BC asks: what would the human do now?

Behavior cloning directly predicts one action and minimizes mean squared error against the demonstrated action. It executes that command, looks again, and repeats. Here the DiT action input and timestep are fixed to zero. The observation condition still changes, so the model can respond to the scene without any diffusion process.

It is called BC because of **how it learns: supervised imitation of demonstrated behavior**. Reusing a Transformer architecture does not make it diffusion. In the broader sense, the diffusion policies also learn by imitation; BC names the direct-regression baseline in this comparison.

Diffusion asks: what short plan fits this view?

During training, add Gaussian noise to a demonstrated 10-action sequence. The network sees the noisy sequence, its noise level, and the observation condition. The epsilon variant learns to predict the added noise; the x0 variant learns to predict the original clean sequence.

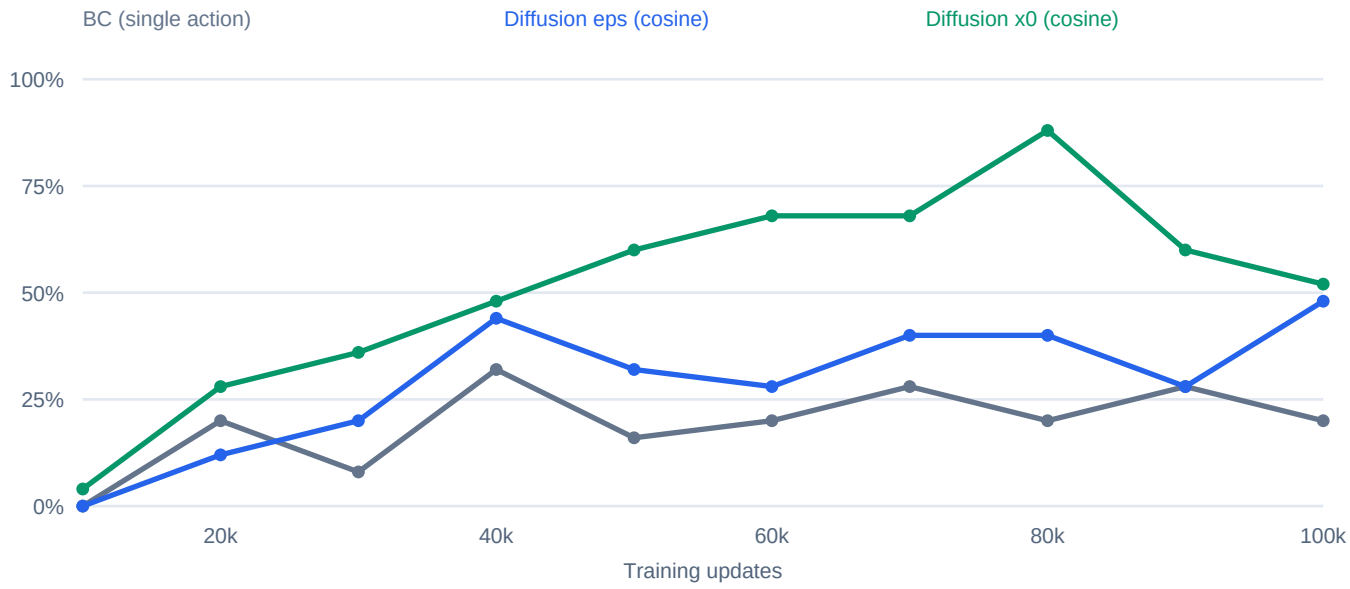
At execution time
Start with random noise -> apply 50 denoising steps -> generate 10 actions -> execute the first 8 -> observe again and replan.

Those 50 steps happen inside the model's sampling loop; they are not 50 robot moves. Executing eight planned actions before replanning is **action chunking**. The architecture aims to represent coherent short motion plans and multiple possible actions, but the measured success rates alone do not prove those mechanisms caused the improvement.

Implementation map: Q3/bc.py; Q3/ddpm.py; Q3/diffusion_policy.py; Q3/networks/; Q3/utils/dataset.py. Backbone: width 320, two blocks, ten attention heads. Predicted actions are unnormalized before simulator execution.

What did the robot learn?

Main experiment | Success rate over training | 25 evaluation episodes per checkpoint



BC	Final: 20%	Peak: 32% at 40k
Diffusion eps / cosine	Final: 48%	Peak: 48% at 100k
Diffusion x0 / cosine	Final: 52%	Peak: 88% at 80k
Bonus: eps / sigmoid	Final: 60%	Peak: 76% at 70k

The diffusion-plus-chunking setup finished ahead of single-action BC. BC ended at 20%, versus 48% and 52% for the two main diffusion variants. This supports the combined design in this run; it does not separate the benefit of diffusion from the benefit of action chunking.

Clean-action prediction peaked higher, then fell. The x0 model reached 88% at 80k updates and finished at 52%. The observed curve is not monotonic. That drop alone does not tell us whether the cause is training instability, evaluation variation, or another mechanism.

The noise schedule mattered in the bonus run. Changing epsilon prediction from cosine to sigmoid ended at 60% rather than 48%, with a peak of 76%. This is a promising setup-specific observation, not a general ranking of schedules.

Configuration from code/script defaults: 100k updates, batch 64, learning rate 1e-4, seed 0, evaluation every 10k, 500-step limit. CSVs record scores and update counts, but do not contain a complete original run manifest. Source: Q3/results/*/eval_metrics.csv.

What can we claim?

A useful robotics result is one we can explain, reproduce, and put in context.

Read the results with the right confidence

There is **one recorded run per method**, with only 25 episodes at each checkpoint. One success changes the score by four percentage points. The final 52% versus 48% gap is just one outcome, so it does not establish that x0 reliably beats epsilon prediction. Peaks were selected from the same evaluation curve and need independent confirmation.

The loader uses all 200 demonstrations, including the dataset's built-in validation subset. Evaluation is simulator execution, not held-out demonstration prediction. BC sums rewards at every step; diffusion sums a maximum within each action chunk. That is why this report compares **task success**, not raw accumulated rewards.

The included videos visualize action-vector denoising, not successful robot rollouts. No failure-category analysis, hardware test, or policy latency benchmark is claimed. An equal number of training updates also does not imply an equal compute budget.

What practical experience does this show?

The work connects visual learning to continuous robot control: preparing demonstrations, encoding two camera views, implementing DDPM objectives and sampling, executing action chunks, integrating MuJoCo evaluation, handling checkpoints, and running GPU jobs. The analysis makes the comparison auditable by retaining evaluation CSVs and deriving the figures from them.

What would I try next?

First, train a chunked BC baseline with the same 10-action horizon and eight-action execution interval. Then repeat all methods across several seeds with larger, separate evaluation sets. Record task rollouts to distinguish grasp failures from alignment failures, and measure inference time. That would tell us more about why performance changes and what still prevents reliable operation.

Explore the project

GitHub: Diffusion Policies for Robot Imitation Learning
Companion project: DDPM math and a short demo

Evidence: Q3/results/{bc,diffusion_policy_eps,diffusion_policy_x0,diffusion_policy_eps_sigmoid}/eval_metrics.csv. Dataset counts: inspected local image.hdf5. Rebuild: scripts/analyze_results.py and scripts/build_report.py. Credits: CSC2626 scaffold; Diffusion Policy (Chi et al.); DDPM (Ho et al., 2020); Robomimic; Robosuite/MuJoCo; CleanDiffuser. Full provenance and reproduction notes are in docs/ATTRIBUTION.md and docs/REPRODUCING.md.